

Hindi Machine Learning Mathematics

Naaranga

September 28 2026

1 Forward propagation

The embedding layer of this network may be represented in terms of matrix multiplication but will be practically handled by `memcpy`ing the relevant vectors into a concatenation layer¹.

The matrix multiplication producing the raw inputs into the first hidden layer is as follows:

$$\begin{bmatrix} w_{1,1} & w_{2,1} & w_{3,1} & \dots & w_{e,1} \\ w_{1,2} & w_{2,2} & w_{3,2} & \dots & w_{e,2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ w_{1,h} & w_{2,h} & w_{3,h} & \dots & w_{e,h} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_e \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_h \end{bmatrix} = \begin{bmatrix} \sum_{n=1}^e w_{n,1}a_n + b_1 \\ \sum_{n=1}^e w_{n,2}a_n + b_2 \\ \vdots \\ \sum_{n=1}^e w_{n,h}a_n + b_h \end{bmatrix} = \begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_h \end{bmatrix}$$

where e is the number of nodes in the embedding layer, h is the number of nodes in the hidden layer, and $\forall x \in \mathbb{N} \cap [1, e]$ and $\forall y \in \mathbb{N} \cap [1, h]$ such that $w_{e,h}$ is the weight linking node e in the embedding layer to node h in the hidden layer, a_e is the activation of node e in the embedding layer, b_h is the bias contributing to node h in the hidden layer, and o_h is the raw output for node h in the hidden layer (before normalization). This can be represented through the following (unoptimized) code based on C syntax:

```
double inputActivations[e], weights[h][e], biases[h],
outputNodes[h] = {0};
// Note that the weights' indices for the hidden layer now
// precedes the indices for the embedding layer.

// The values of inputActivations, weights, and biases would be
// computed through some algorithm here.

// Calculate the values for the set o:
for (int i = 0; i < h; i++) {
    for (int j = 0; j < e; j++)
```

¹Refer to the neural network flowchart and `naiveNetwork.c` in this project's GitHub repository for conceptual and programmatic realizations, respectively, of this algorithm.

```

        outputNodes[i] = weights[i][j]*inputActivations[j];
    outputNodes[i] += biases[i];
}

```

This operation can then be generalized to any two consecutive hidden layers if e and h are taken to be the number of nodes in the previous and next hidden layers, respectively.

ReLU will then be utilized as a normalization function to compute the activation of the next nodes:

$$A_n = \max(0, o_n) = \begin{cases} 0 & \text{if } o_n \leq 0 \\ o_n & \text{if } o_n > 0 \end{cases}$$

where $n \in \mathbb{N} \cap [1, h]$ such that A_n is the final activation of node h in the next hidden layer. In C, this may be implemented through a simple ternary operator such as $(oN \leq 0) ? 0 : oN$, where oN represents o_n as a programmatic variable². This methodology additionally drastically simplifies the calculus necessary for backpropagation:

$$\frac{\delta A_n}{\delta o_n} = \begin{cases} 0 & \text{if } o_n < 0 \\ 1 & \text{if } o_n > 0 \end{cases}$$

The use of lowercase delta to indicate partial differentiation is consistent with conventional chain rule representations of the computation involved in simple neural network backpropagation.

²The WPILib convention for incorporating mathematical variables with subscripts in Java code, in which the base variable is lowercase, and the subscript is capitalized, is adopted here.